

Docker & Docker Compose

Guía de preparación para entrevista técnica
Fullstack · BBVA

Proyecto analizado:

Ventas-Docker / docker-compose.yml

ERP de Ventas – PostgreSQL 16 + Spring Boot (backend) + Angular (frontend)

Contenido: fundamentos de Docker, comandos esenciales, análisis del docker-compose del proyecto, redes, volúmenes, Dockerfile, buenas prácticas y preguntas frecuentes de entrevista con respuestas.

Índice

- 1. Análisis del docker-compose.yml del proyecto
- 2. Fundamentos de Docker: contenedores, imágenes y capas
- 3. Comandos esenciales de Docker
- 4. Docker Compose: comandos y claves del archivo
- 5. Dockerfile: cómo se construye una imagen
- 6. Redes en Docker
- 7. Volúmenes y persistencia de datos
- 8. Buenas prácticas y seguridad
- 9. Preguntas frecuentes de entrevista (con respuestas)
- 10. Cheat sheet rápido de comandos

Palabras clave para la entrevista: contenedor, imagen, capa, registry, orquestación, nombre-de-contenedor, DNS interno, bind mount, named volume, healthcheck, multi-stage build, 12-factor.

1. Análisis del docker-compose.yml del proyecto

Este es el archivo del repositorio **Ventas-Docker**. Levanta el sistema completo del ERP de Ventas con tres servicios.

```
services:
  postgres:
    image: postgres:16
    container_name: erp-postgres
    environment:
      POSTGRES_DB: ventas_db
      POSTGRES_USER: postgres
      POSTGRES_PASSWORD: admin
    ports:
      - "5432:5432"
    volumes:
      - postgres_data:/var/lib/postgresql/data

  backend:
    build:
      context: ../Ventas-Backend
    container_name: erp-backend
    ports:
      - "8081:8081"
    depends_on:
      - postgres
    environment:
      SPRING_DATASOURCE_URL: jdbc:postgresql://postgres:5432/ventas_db
      SPRING_DATASOURCE_USERNAME: postgres
      SPRING_DATASOURCE_PASSWORD: admin
      SERVER_PORT: 8081

  frontend:
    build:
      context: ../Ventas-Frontend
    container_name: erp-frontend
    ports:
      - "4200:80"
    depends_on:
      - backend

volumes:
  postgres_data:
```

1.1 Desglose de cada servicio

Servicio	Imagen/base	Puerto host:contenedor	Persistencia
postgres	postgres:16	5432 → 5432	volumen postgres_data
backend	build ../Ventas-Backend	8081 → 8081	—
frontend	build ../Ventas-Frontend	4200 → 80	—

1.2 Comunicación entre servicios

Compose crea una **red bridge por defecto**. Dentro de esa red, cada servicio se resuelve por su nombre DNS: **postgres**, **backend**, **frontend**.

El backend usa `jdbc:postgresql://postgres:5432/ventas_db`: no usa `localhost` porque `localhost` dentro del contenedor apunta al propio contenedor, no al host ni a otro contenedor.

```
Flujo de peticiones:
Navegador (http://localhost:4200)
→ frontend (nginx, puerto 80 interno, expuesto como 4200)
→ backend (Spring Boot, puerto 8081)
→ postgres (PostgreSQL 16, puerto 5432)
```

1.3 Claves a defender en la entrevista

- **image:** usa la imagen oficial postgres:16 de Docker Hub. El backend y frontend se **construyen** (build) desde sus propios repositorios con un Dockerfile.
- **build.context:** apunta fuera de este repo (../Ventas-Backend y ../Ventas-Frontend). Compose toma el Dockerfile de esa carpeta.
- **container_name:** fija el nombre exacto del contenedor para que sea predecible en la red.
- **environment:** variables de entorno. En un proyecto real irían en un archivo .env o secretos, nunca hardcodeadas (aquí aparece admin como password).
- **ports:** publica el puerto del contenedor en el host con formato host:contenedor.
- **depends_on:** define orden de arranque. Ojo: solo espera a que el contenedor se cree/arranque, NO a que la app esté lista (para eso se usa healthcheck).
- **volumes / postgres_data:** named volume que persiste los datos de PostgreSQL aunque el contenedor se elimine o reconstruya.

Pregunta típica: ¿qué harías para que el backend no se caiga si PostgreSQL tarda en estar listo?
 Respuesta: añadir dependencia a un **healthcheck** de postgres y usar depends_on: condition: service_healthy (Docker Compose v2).

2. Fundamentos de Docker: contenedores, imágenes y capas

2.1 ¿Qué es Docker?

Docker es una plataforma de **contenedores** que empaqueta una aplicación con todas sus dependencias (código, runtime, librerías, configuración) en unidades aisladas y portables. Garantiza que funcione igual en desarrollo, pruebas y producción.

2.2 Contenedor vs Máquina Virtual

Criterio	Contenedor	Máquina virtual (VM)
Aislamiento	Comparte el kernel del host	Kernel propio (hipervisor)
Tamaño	MB / decenas de MB	GB
Arranque	Milisegundos / segundos	Minutos
Recursos	Liviano, alto rendimiento	Mayor uso de CPU/RAM
SO invitado	No incluye SO completo	SO completo por VM
Uso típico	Microservicios, apps cloud-native	Legacy, entornos con SO distintos

Frase para la entrevista: “Un contenedor es un proceso aislado que comparte el kernel del host con namespace namespaces y control groups (cgroups) para limitar recursos”.

2.3 Imágenes y capas (layers)

- **Imagen:** plantilla inmutable de solo lectura con el código y dependencias. Se define en un Dockerfile.
- **Contenedor:** instancia en ejecución de una imagen. Añade una capa de escritura sobre la imagen.
- **Capa (layer):** cada instrucción de un Dockerfile crea una capa. Las capas se cachean: si no cambia una instrucción y las anteriores, Docker reutiliza esa capa (cache).
- **Registry:** repositorio de imágenes. Público: Docker Hub. Privado: Amazon ECR, Azure ACR, Google Artifact Registry, Harbor, Nexus.
- **Dockerfile:** receta de construcción. Con `docker build` se obtiene una imagen.

ONU (Overlay filesystem): la imagen es read-only y el contenedor usa un filesystem en capas; los cambios del contenedor se pierden si el contenedor se elimina salvo que se guarden en volúmenes.

2.4 Arquitectura cliente-servidor de Docker

```
docker CLI (tu terminal)
↔ daemon dockerd (el motor)
↔ containerd / runc (runtime de contenedores)
↔ imágenes en el host / registry
```

- **CLI:** comandos que ejecutas (`docker run`, `docker build`, `docker compose`).
- **Daemon (dockerd):** proceso que construye, ejecuta y administra contenedores.
- **containerd:** runtime que gestiona el ciclo de vida del contenedor.
- **runc / OCI:** estándar de contenedores compatible con AWS, Kubernetes, etc.

3. Comandos esenciales de Docker

3.1 Imágenes

- **docker images** — Lista las imágenes descargadas localmente.
- **docker pull nombre:tag** — Descarga una imagen del registry. Ej: `docker pull postgres:16`
- **docker rmi nombre** — Elimina una imagen.
- **docker build -t mi_app:1.0 .** — Construye una imagen a partir del Dockerfile del directorio.
- **docker tag img repo:tag** — Etiqueta una imagen para un registro.
- **docker push repo:tag** — Sube una imagen a un registry.
- **docker image prune** — Elimina imágenes colgantes (dangling).

3.2 Contenedores

- **docker run -d -p 8081:8081 --name erp-backend mi_backend** — Crea y arranca un contenedor en segundo plano (-d) publicando el puerto y asignando nombre.
- **docker ps** — Lista contenedores en ejecución.
- **docker ps -a** — Lista todos (incluye parados).
- **docker start / stop / restart nombre** — Gestiona el ciclo de vida.
- **docker rm nombre** — Elimina un contenedor (debe estar parado).
- **docker logs -f erp-backend** — Muestra el stdout/stderr del contenedor en vivo.
- **docker exec -it erp-backend /bin/bash** — Entra al shell dentro del contenedor en ejecución.
- **docker inspect erp-backend** — Muestra toda la config en JSON (IP, redes, env, mounts).
- **docker cp archivo contenedor:/ruta** — Copia archivos entre host y contenedor.
- **docker stats** — Monitoriza CPU/RAM/red de los contenedores.

3.3 Red, volúmenes y limpieza

- **docker network ls / create / inspect** — Administra redes.
- **docker volume ls / create / inspect** — Administra volúmenes.
- **docker system prune -a --volumes** — Limpieza total: contenedores parados, redes e imágenes no usadas, volúmenes sin usar. CUIDADO: borra volúmenes.
- **docker info** — Información del motor (versión, runtime, drivers).
- **docker version** — Versiones de cliente y demonio.

Ejemplo completo para este proyecto: `docker compose up -d` → levanta los 3 servicios en background
`docker compose logs -f backend` → sigue los logs del backend `docker exec -it erp-postgres bash` → entra a la base de datos `docker compose down -v` → baja todo y ELIMINA los volúmenes

4. Docker Compose: comandos y claves del archivo

Docker Compose define y gestiona aplicaciones **multi-contenedor** con un archivo `docker-compose.yml` (YAML). Es la herramienta usada en este proyecto para orquestar el ERP completo con un solo comando.

4.1 Comandos de Compose

- **docker compose up -d** — Construye (si hace falta) y levanta los servicios en segundo plano.
- **docker compose up -d --build** — Fuerza la reconstrucción de imágenes (uso frecuente tras cambios de código).
- **docker compose build** — Construye las imágenes sin levantar servicios.
- **docker compose ps** — Estado de los servicios del proyecto.
- **docker compose logs -f servicio** — Logs en vivo. Servicio: postgres, backend o frontend.
- **docker compose exec servicio comando** — Ejecuta un comando dentro de un servicio en marcha. Ej: `docker compose exec backend ls`
- **docker compose down** — Detiene y elimina contenedores y red del proyecto.
- **docker compose down -v** — Igual que down pero también elimina los volúmenes (pierde datos).
- **docker compose config** — Valida el archivo y muestra la config resuelta.
- **docker compose pull** — Descarga las imágenes sin crearlas.
- **docker compose restart** — Reinicia los servicios.
- **docker compose top** — Lista los procesos internos de cada servicio.

4.2 Claves principales de docker-compose.yml

Clave	Significado	Uso en este proyecto
version	Versión del schema (obsoleta en v2, se omite).	No aparece (correcto en Compose v2).
services	Define cada contenedor/componente.	postgres, backend, frontend.
image	Imagen predefinida del registry.	postgres:16.
build	Dockerfile con el que construir la imagen.	context: ../Ventas-Backend y ../Ventas-Frontend.
container_name	Nombre fijo del contenedor.	erp-postgres, erp-backend, erp-frontend.
environment	Variables de entorno del contenedor.	credenciales BD y SERVER_PORT.
ports	Publicación host:contenedor.	5432:5432, 8081:8081, 4200:80.
volumes	Monto los volúmenes en el contenedor.	postgres_data:/var/lib/postgresql/data.
depends_on	Orden de arranque entre servicios.	backend ← postgres; frontend ← backend.
networks	Redes personalizadas (opcional).	No definida: usa la red por defecto.
healthcheck	Comando que comprueba que el servicio está listo.	No definido (añadir como mejora).

restart	Política de reinicio (no, always, on-failure).	No definida (añadir como mejora).
env_file	Carga variables de un archivo .env.	No usado (mejora: no hardcodear secretos).

4.3 dependencias y orden de arranque

`depends_on` garantiza orden de creación/arranque, no que el servicio responda. En Compose v2 se puede usar:

```
depends_on:
  postgres:
    condition: service_healthy

postgres:
  healthcheck:
    test: ["CMD-SHELL", "pg_isready -U postgres"]
    interval: 10s
    retries: 5
    start_period: 5s
```

5. Dockerfile: cómo se construye una imagen

Un Dockerfile es un archivo de texto con las instrucciones para construir una imagen capa a capa. En este repositorio los Dockerfiles están fuera (en Ventas-Backend y Ventas-Frontend) y Compose los invoca con **build**.

5.1 Instrucciones principales

- **FROM imagen:tag** — Define la imagen base. Debe ser la primera instrucción.
- **RUN comando** — Ejecuta comandos en la construcción (instalar paquetes, compilar, etc.).
- **COPY origen destino** — Copia archivos del contexto de build al contenedor.
- **ADD origen destino** — Como COPY pero permite URLs y descomprimidos (evitar si no hace falta).
- **WORKDIR /ruta** — Directorio de trabajo; cambia el CWD para las siguientes instrucciones.
- **ENV VAR=valor** — Define variables de entorno (persisten en el contenedor).
- **ARG nombre** — Variable solo durante el build (no persiste en la imagen).
- **EXPOSE puerto** — Documenta qué puerto usa la app (no lo publica; eso es -p o ports).
- **CMD ["..."]** — Comando por defecto al arrancar el contenedor. Solo una CMD real (se sobrescribe con docker run args).
- **ENTRYPOINT ["..."]** — Comando fijo; CMD le pasa argumentos. Combinados definen el proceso principal (PID 1).
- **USER usuario** — Ejecuta el proceso con usuario no root (seguridad).
- **HEALTHCHECK** — Comando de salud que Docker ejecuta periódicamente.
- **LABEL** — Metadatos de la imagen (autor, versión).

5.2 Ejemplos prácticos para el proyecto

Backend (Java / Spring Boot) — build con Maven y capa runtime

```
# Multi-stage build
FROM maven:3.9-eclipse-temurin-17 AS build
WORKDIR /app
COPY pom.xml .
RUN mvn dependency:go-offline
COPY src ./src
RUN mvn clean package -DskipTests

FROM eclipse-temurin:17-jre
WORKDIR /app
COPY --from=build /app/target/*.jar app.jar
EXPOSE 8081
ENTRYPOINT ["java", "-jar", "app.jar"]
```

Frontend (Angular) — build de Node y servir con nginx

```
FROM node:20-alpine AS build
WORKDIR /app
COPY package*.json ./
RUN npm ci
COPY . .
RUN npm run build --configuration=production

FROM nginx:alpine
```

```
COPY --from=build /app/dist/ventas /usr/share/nginx/html
COPY nginx.conf /etc/nginx/conf.d/default.conf
EXPOSE 80
CMD ["nginx", "-g", "daemon off;"]
```

PostgreSQL

```
FROM postgres:16
ENV POSTGRES_DB=ventas_db
ENV POSTGRES_USER=postgres
# Los scripts en /docker-entrypoint-initdb.d se ejecutan al primer arranque
COPY init.sql /docker-entrypoint-initdb.d/
```

5.3 Diferencias clave

Instrucción	Diferencia
CMD vs ENTRYPOINT	CMD se puede sobrescribir al correr; ENTRYPOINT no (es el proceso fijo). Juntos: ENTRYPOINT = binario, CMD = argumentos por defecto.
COPY vs ADD	COPY solo copia archivos; ADD además soporta URLs y descomprime tarballs (preferir COPY por predictibilidad).
RUN vs CMD	RUN se ejecuta durante el build (instala, compila); CMD se ejecuta al arrancar el contenedor.
Multistage	Varias etapas FROM: se usa una etapa para compilar y otra más liviana para ejecutar → imágenes pequeñas.

La orden de las instrucciones importa para la cache: coloca antes lo que cambia menos (dependencias) y al final el código, para aprovechar las capas cacheadas y acelerar los builds.

6. Redes en Docker

6.1 Tipos de red (driver)

Driver	Descripción	¿Cuándo?
bridge (default)	Red privada del host; los contenedores se comunican entre sí y con el exterior solo por los puertos publicados.	Aplicaciones en un solo host. Es la que usa Compose por defecto.
host	El contenedor comparte la red del host (sin NAT). Aislar puertos es más rápido pero sin aislamiento.	Apps que requieren rendimiento de red o puertos dinámicos.
none	Sin red; solo interfaz loopback.	Tareas aisladas de red.
overlay	Red multi-host, usada por Swarm/Kubernetes.	Clusters, orquestación.

6.2 Puntos clave para la entrevista

- **DNS interno:** en una red de Compose los contenedores se resuelven por su nombre de servicio. Por eso el backend usa postgres como host de la BD.
- **Publicación de puertos:** ports: "4200:80" significa que el puerto 4200 del host redirige al 80 del contenedor. Sin publicación, solo otros contenedores de la misma red pueden alcanzarlo.
- **Bridge por defecto vs user-defined bridge:** Compose crea una red bridge definida por el usuario, que sí ofrece resolución DNS automática entre contenedores.
- **Aislamiento:** cada proyecto Compose crea una red; contenedores de otros proyectos no se ven entre sí (a menos que compartan red).

```
Comandos red del proyecto:
docker network ls
docker network inspect # ver IPs y miembros
docker compose down # elimina la red creada por Compose
```

En Kubernetes, el equivalente del DNS por nombre de servicio lo resuelve el **Service** y el **Service DNS** (mi-servicio.namespace.svc.cluster.local). Es una buena transición para mencionar si preguntan por orquestación.

7. Volúmenes y persistencia de datos

El sistema de archivos de un contenedor es **efímero**: se pierde al eliminar el contenedor. Los volúmenes persisten datos fuera del ciclo de vida del contenedor.

7.1 Tipos de almacenamiento

Tipo	Dónde vive	Uso	Ejemplo
Named volume	Gestionado por Docker (/var/lib/docker/volumes)	Datos persistentes de apps y BBDD.	postgres_data:/var/lib/postgresql/data
Bind mount	Cualquier carpeta del host	Código en desarrollo (hot reload), config.	./src:/app/src
tmpfs	En memoria del host	Datos temporales que no deben persistir.	--tmpfs /tmp

7.2 En este proyecto

- El bloque final volumes: postgres_data: declara un **named volume** llamado postgres_data.
- Se monta en /var/lib/postgresql/data, el directorio donde PostgreSQL guarda sus datos. Así, aunque hagas docker compose down (sin -v) y vuelvas a up, los datos se conservan.
- docker compose down -v ELIMINA el volumen y por tanto toda la información de la BD. Ojo al usarlo en entornos reales.
- En producción con Kubernetes se usaría un **PersistentVolumeClaim (PVC)** con un StorageClass (EBS/GCE/PD), no un volumen local.

```
Comandos útiles:
docker volume ls
docker volume inspect postgres_data
docker volume prune # elimina volúmenes no usados
```

8. Buenas prácticas y seguridad

8.1 Imágenes y Dockerfile

- Usar imágenes base oficiales y con tag explícito (postgres:16, node:20-alpine, eclipse-temurin:17-jre). Evitar latest en producción.
- Multi-stage builds para imágenes pequeñas: compilar en una etapa, ejecutar en otra con solo JDK/JRE y artefactos.
- Reducir el número de capas y ordenarlas por frecuencia de cambio (dependencias antes que código) para aprovechar la cache de build.
- Usar imágenes **alpine** o distroless para reducir superficie.
- No guardar secretos en la imagen: usar variables de entorno, env_file o secretos del orquestador.
- Ejecutar como usuario no root (USER node / USER 1000).
- Añadir **HEALTHCHECK** en servicios críticos (BD, API).
- Escaneo de vulnerabilidades: docker scout (o Trivy, Snyk, Gype) en CI/CD.

8.2 Compose y operación

- No hardcodear contraseñas: usar env_file: .env y que .env no se committee (añadirlo a .gitignore).
- Definir **restart: unless-stopped** o **on-failure** para servicios de producción.
- Publicar solo los puertos realmente necesarios (frontend 4200, backend 8081; la BD no debería exponerse al host en producción).
- Usar logging con límites (logging.driver, max-size/max-file) para no llenar el disco.
- Conocer el **principio de mínimo privilegio**: contenedores solo con lo necesario.

Precaución con `docker system prune -a`: borra imágenes, redes y la cache de build. Añade `--volumes` y además borrará datos persistentes. En entornos compartidos, confirmar antes.

9. Preguntas frecuentes de entrevista (con respuestas)

¿En qué se diferencia Docker de una VM?

Respuesta: El contenedor comparte el kernel del host con namespaces y cgroups; es más ligero y arranca casi al instante. La VM virtualiza el hardware con un kernel propio, por lo que es más pesada e aislada.

¿Cómo funciona la cache de capas del build?

Respuesta: Cada instrucción del Dockerfile genera una capa. Si una capa no cambia respecto al build anterior (mismo contexto), Docker la reutiliza. Por eso conviene copiar dependencias antes que el código.

¿Diferencia entre CMD y ENTRYPOINT?

Respuesta: CMD define argumentos por defecto y es sobrescribible al ejecutar docker run; ENTRYPOINT es el proceso fijo. Se combinan: ENTRYPOINT el binario y CMD los argumentos por defecto.

¿Cómo se comunican los contenedores entre sí en Compose?

Respuesta: A través de la red bridge que crea Compose, usando el nombre del servicio como DNS (jdbc:postgresql://postgres:5432/ventas_db). El DNS lo resuelve el engine de Docker.

¿Qué hace depends_on y qué limitación tiene?

Respuesta: Ordena el arranque de servicios, pero no espera a que la app esté lista (solo a que el contenedor esté creado/arrancado). Para esperar el estado real se usa healthcheck + condition: service_healthy.

¿Cómo persistir los datos de la base de datos?

Respuesta: Con un named volume montado en el directorio de datos de PostgreSQL. En el proyecto: postgres_data:/var/lib/postgresql/data. En k8s, con un PVC.

¿Por qué el frontend expone 4200 pero dentro usa el 80?

Respuesta: Porque la imagen de frontend es nginx, que escucha en el puerto 80 dentro del contenedor. El 4200 solo es el puerto público del host hacia el 80 interno (compatibilidad de Angular CLI).

¿Exponer los puertos en Compose es seguro en producción?

Respuesta: Lo ideal es no publicar puertos de todos los servicios: el frontend y el backend pueden comunicarse por la red interna, y exponer solo un punto (por ejemplo el frontend). Los servicios internos se alcanzan por nombre DNS.

¿Qué es un multi-stage build y por qué usarlo?

Respuesta: Un Dockerfile con varios FROM: una etapa compila (Maven/npm) y una segunda etapa solo copia los artefactos a una imagen base más ligera. Resultado: menor tamaño y menor superficie de ataque.

¿Cómo verificar los logs o entrar a un contenedor en ejecución?

Respuesta: docker compose logs -f backend para logs en vivo y docker exec -it erp-backend bash para abrir un shell. docker inspect erp-backend para ver la config completa (redes, env, IP).

¿Qué diferencia hay entre una imagen y un contenedor?

Respuesta: La imagen es el artefacto inmutable (plantilla) con el código y las dependencias. El contenedor es la instancia en ejecución, con su capa de escritura y su propio PID/red/fs aislados.

¿Cómo limitar el consumo de recursos?

Respuesta: Con la sección deploy o docker run --memory, --cpus. en Compose: deploy.limits.cpus y deploy.limits.memory; en la práctica docker daña el contenedor si supera los límites.

¿Se pueden usar variables de entorno en Compose?

Respuesta: Sí, con \${VAR} en el YAML, sustituidas desde el archivo .env o el shell. Es la forma correcta de no hardcodear secretos (a diferencia de la clave environment literal del compose actual).

¿Qué es un healthcheck?

Respuesta: Un comando que Docker ejecuta periódicamente (interval) para conocer el estado del contenedor (healthy/unhealthy). Permite a orquestadores y a depends_on reaccionar al estado real del servicio.

¿Qué herramientas conoces para escalar contenedores?

Respuesta: Kubernetes como orquestador de referencia, Docker Swarm como opción simple dentro del stack Docker, y plataformas como Rancher, OpenShift, ECS/EKS/AKS. En BBVA es habitual hablar de Kubernetes y entornos on-prem/cloud (OpenShift/ASI).

¿Qué es un contenedor PID 1 y por qué importa?

Respuesta: El proceso principal del contenedor (CMD/ENTRYPOINT) es el PID 1: recibe las señales del sistema (SIGTERM/SIGKILL) y las orquesta. Si no maneja señales, el stop limpio fallará y se usará kill forzado.

¿Cómo harías CI/CD para esta app?

Respuesta: Jenkins/GitLab CI/Azure DevOps: build de las imágenes con docker build, escaneo de seguridad, push a un registry (ACR/ECR/Harbor), despliegue con docker compose o Helm en Kubernetes, y healthchecks después del deploy.

Consejo: además de responder, relaciona cada tema con este proyecto concreto (explícale al entrevistador qué hace cada servicio del docker-compose y cómo lo mejorarías: healthchecks, secretos, restricción de puertos).

10. Cheat sheet rápido de comandos

Gestión básica

\$ docker ps -a
\$ docker images
\$ docker info
\$ docker version

Ciclo de vida

\$ docker run -d -p 8081:8081 --name app img:tag
\$ docker start stop restart nombre
\$ docker rm -f nombre
\$ docker kill nombre

Inspección y logs

\$ docker logs -f nombre
\$ docker logs --tail 100 nombre
\$ docker inspect nombre
\$ docker stats
\$ docker exec -it nombre /bin/sh

Imágenes / build

\$ docker build -t nombre:tag .
\$ docker pull push nombre:tag
\$ docker rmi nombre:tag
\$ docker image prune -a

Recursos

\$ docker volume ls create inspect prune
\$ docker network ls create inspect prune
\$ docker system df
\$ docker system prune (opcional: -a --volumes)

Compose

\$ docker compose up -d --build
\$ docker compose ps
\$ docker compose logs -f
\$ docker compose exec servicio comando
\$ docker compose config

```
$ docker compose down
```

```
$ docker compose down -v
```

Modo de depuración de 4 pasos en entrevista: 1) docker ps -a ¿está corriendo? 2) docker logs ¿qué dice la app? 3) docker inspect ¿config/env/red correctas? 4) docker exec para probar conectividad dentro del contenedor.